18.3 区间DP

18.3.1 定义

以区间大小变化为阶段的DP问题

18.3.2 特点

- 区间DP是线性DP的拓展，它的状态是一段或者是平面中的一片
- 从小区间求到大区间，小区间合并取优得到大区间的最优值
- 状态转移就可以看成区间合并，有些是左右合并，有些是枚举断开点合并
- 数组下标描述区间范围，数组值表示最优值
- 到底用递推实现还是记忆化搜索实现，视题目而定

用几道例题来体现上述特点

例题18-28：两端取数 (POJ3186)

【题目描述】
有n个数，每次只能从两端取，当次取出的价值为数的价值乘以他被取出的顺序号，求所有数的最大价值和。
【样例输入】

```
5
1
3
1
5
2
```

【样例输出】

```
43
```

【分析】

模拟解决样例的过程，我们用a[1]~a[5]的表示 1 3 1 5 2

用带括号数字（1 3 1 5 2）表示取完a[1]~a[5]所能获得的最大价值和

那么（1 3 1 5 2）的求取可以分两种情况，取更优者，如图18-1

- 先取a[1]，再取a[2]~a[5]，最优值为：（3 1 5 2）+ 1*1
- 先取a[5]，再取a[1]~a[4]，最优值为：（1 3 1 5）+ 2*1

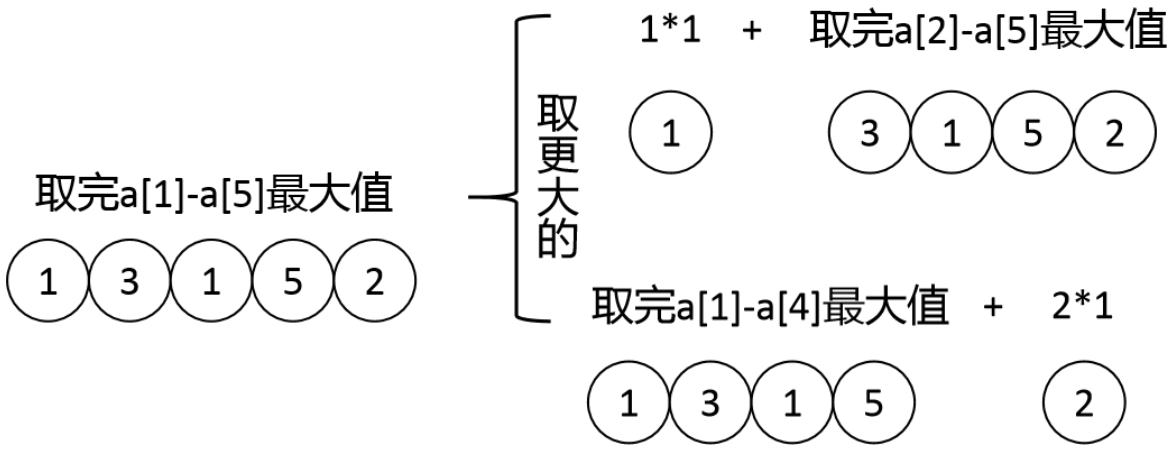


图 18 – 1

（3 1 5 2）的求取又分为两种情况，取更优者

- 先取a[2]，再取a[3]~a[5]，最优值为：（1 5 2）+ 3*2
- 先取a[5]，再取a[2]~a[4]，最优值为：**(3 1 5)** + 2*2

（1 3 1 5）的求取又分为两种情况，取更优者

- 先取a[1]，再取a[2]~a[5]，最优值为：**(3 1 5)** + 1*2
- 先取a[4]，再取a[1]~a[3]，最优值为：（1 3 1）+ 5*2

同样原理可以继续深入求解：

（1 5 2）的来源：（1 5）+ 2*3 或 （5 2 ）+ 1*3

（3 1 5）的来源：（3 1）+ 5*3 或 （1 5） + 3*3

（1 3 1）的来源：（3 1）+ 1*3 或 （1 3） + 1*3

（1 5）的来源：（5）+ 1*4 或 （1）+ 5*4

（5 2）的来源：（2）+ 5*4 或 （5）+ 2*4

（3 1）的来源：（1）+ 3*4 或 （3）+ 1*4

以上过程是从大区间递归到小区间，类似于搜索的过程，注意加粗部分**(3 1 5)**，出现了重复调用，可以用**记忆化搜索**避免

记忆化搜索实现代码

```
#include<cstdio>
#include<algorithm>
using namespace std;
int dp[2100][2100],a[2100],n;
void dfs(int i,int j){
    if (dp[i][j]) return;//已搜索过的状态值放在数组中，避免重复搜索，就是记忆化搜索
    if (i==j) {
        dp[i][j]=a[i]*n;
        return;
    }
    dfs(i+1,j);
    dfs(i,j-1);
    dp[i][j]=max(dp[i+1][j]+a[i]*(n-j+i),dp[i][j-1]+a[j]*(n-j+i));
}
int main(){
    scanf("%d",&n);
    for (int i=1;i<=n;i++)
```

```
        scanf("%d",&a[i]);
    dfs(1,n);
    printf("%d\n",dp[1][n]);
}
```

此外，我们也可以用**递推**的方法来做，要改变求解的顺序，先求（1）、（3）、（1）、（5）、（2）这些取一个元素的最优值，再求（1 3）、（3 1）、（1 5）、（5 2）这些取两个元素的最优值，再求（1 3 1）、（3 1 5）、（1 5 2）这些取三个元素的最优值，再求（1 3 1 5 ）、（3 1 5 2）这些取四个元素的最优值，再最终求出（1 3 1 5 2）取五个元素的最优值

无论**记忆化搜索**还是**递推**，都用到了形如（1 3 1 5 2）的带括号数字，括号内的数字表示要取数的区域，括号表示最优值，其实就是动态规划里的**状态**，我们用数组 $dp[i][j]$ 来描述，它表示取完第i个到第j个元素的最优值，那么（1 3 1 5 2）就可以表示成 $dp[1][5]$

如何求 $dp[i][j]$ 呢，根据上面的求解过程，可以进行归纳， $dp[i][j]$ 就是如下的更优者

- $dp[i+1][j]+a[i]*(n-j+i)$
- $dp[i][j-1]+a[j]*(n-j+i)$

接着就是求解顺序，按照区间长度升序求解

长度为1：$dp[1][1]$、$dp[2][2]$ 、$dp[3][3]$ 、$dp[4][4]$ 、$dp[5][5]$

长度为2：$dp[1][2]$、$dp[2][3]$ 、$dp[3][4]$ 、$dp[4][5]$

长度为3：$dp[1][3]$、$dp[2][4]$、$dp[4][5]$

长度为4：$dp[1][4]$、$dp[2][5]$

长度为5：$dp[1][5]$

我们把求解所有状态的过程叫做**状态转移**

状态转移的代码：

```
for (int L=2;L<=n;L++)//升序枚举长度，先求小区间再求大区间，因为大区间依赖小区间
    for (int i=1;i<=n-L+1;i++){//枚举区间的头部i,由于区间有长度，所以结尾是n-L+1
        int k=i+L-1;//区间的尾部k根据长度和头部就可以计算出来
        dp[i][j]=max(dp[i+1][j]+a[i]*(n-j+i),dp[i][j-1]+a[j]*(n-j+i));//dp[i][j]
的两个来源
    }
```

上述代码并没有遍历长度为1的状态，因为长度为1的状态值不求也能得到，我们把这种值称为**初值**，求**初值**代码如下：

```
for(int i=1;i<=n;i++) dp[i][i]=a[i]*n;//只剩一个元素的时候，显然它是第n个，要乘以n
```

总结上述区间DP的概念：

状态：$dp[i][j]$ 表示取从 i 到 j 的数能够获得的最大价值

状态转移：$dp[i][j]=max\{dp[i+1][j]+a[i]*(n-j+i),dp[i][j-1]+a[j]*(n-j+i)\}$

初值：$dp[i][i]=a[i]*n$

终值：$dp[1][n]$

时间复杂度： $O(n^2)$

【递推实现代码】

```
#include<bits/stdc++.h>
using namespace std;
int n,a[2010],dp[2010][2010];
int main(){
    scanf("%d",&n);
    for(int i=1;i<=n;i++) scanf("%d",&a[i]);
    for(int i=1;i<=n;i++) dp[i][i]=a[i]*n;
    for(int L=2;L<=n;L++)
        for(int i=1;i<=n-L+1;i++){
            int j=i+L-1;
            dp[i][j]=max(dp[i+1][j]+a[i]*(n-j+i),dp[i][j-1]+a[j]*(n-j+i));
        }
    printf("%d\n",dp[1][n]);
}
```

例题18-29：牛吃草（POJ3042）

【题目描述】

$n\ (1 \le n \le 1000)$ 棵草排成一线，牛从某个指定位置，开始只能往就近的两边吃，吃不花时间，允许来回，速度恒定，草有个变质度，随时长加剧，求最小变质度

草的位置$L\ (1 \le L \le 1000000)$

【输入】

n 和牛的初始位置

【输出】

```
4 10
1
9
11
19
```

【样例输出】

```
44
```

【样例解释】

牛于 0 时刻在 10 号位，1 时刻到达 9 号位，3 时刻到达 11 号位，11 时刻到达 19 号位，29 时刻到达 1 号位，总共产生 44 的变质度。

【题目分析】

和上一题相比

差异一：此题从中间往两边扩宽，上一题是两边往中间收紧

差异二：我们不但要关注目前已经吃完了哪些草（例题1中取了哪一段的数），而且要关注牛所在的位置（例题1中可以任意取头或尾的数，不用关心上一次取到了哪里），因为牛的移动距离直接影响着我们的求解，这其实就是要遵循动态规划里的基本原则：**无后效性**，详细来说就是某阶段的状态一旦确定，则此后过程的演变不再受此前各状态及决策的影响。

另外请注意给定的草的位置不一定是有序的

如何解决这两个差异，我们可以调整**状态**：

$dp[i][j][1]$表示牛吃完第i到j棵草，并且牛在j位置时，已产生的最小变质度和

$dp[i][j][0]$表示牛吃完第i到j棵草，并且牛在i位置时，已产生的最小变质度和

状态转移也随之调整：

$$dp[i][j][0] = min(dp[i+1][j][0] + (a[i+1] - a[i])(n-j+i), dp[i+1][j][1] + (a[j] - a[i])(n-j+i));$$

$$dp[i][j][1] = min(dp[i][j-1][1] + (a[j] - a[j-1])(n-j+i), dp[i][j-1][0] + (a[j] - a[i])(n-j+i));$$

初值

$dp[\text{起点}][\text{起点}][0] = 0; dp[\text{起点}][\text{起点}][1] = 0$,其他赋最大值

终值

$min(dp[1][n][0], dp[1][n][1])$

区间DP递推写法也有多种，主要是区间求取顺序的差异，如下图两种方式都是可以的，唯一的原则就是大区间所依赖的小区间要先求出来，无论是记忆化搜索还是递推都要遵循这一原则。

时间复杂度

$O(n^2)$

类似上一题递推写法的代码

这种写法的区间求解顺序如图 $18-2$ 所示：

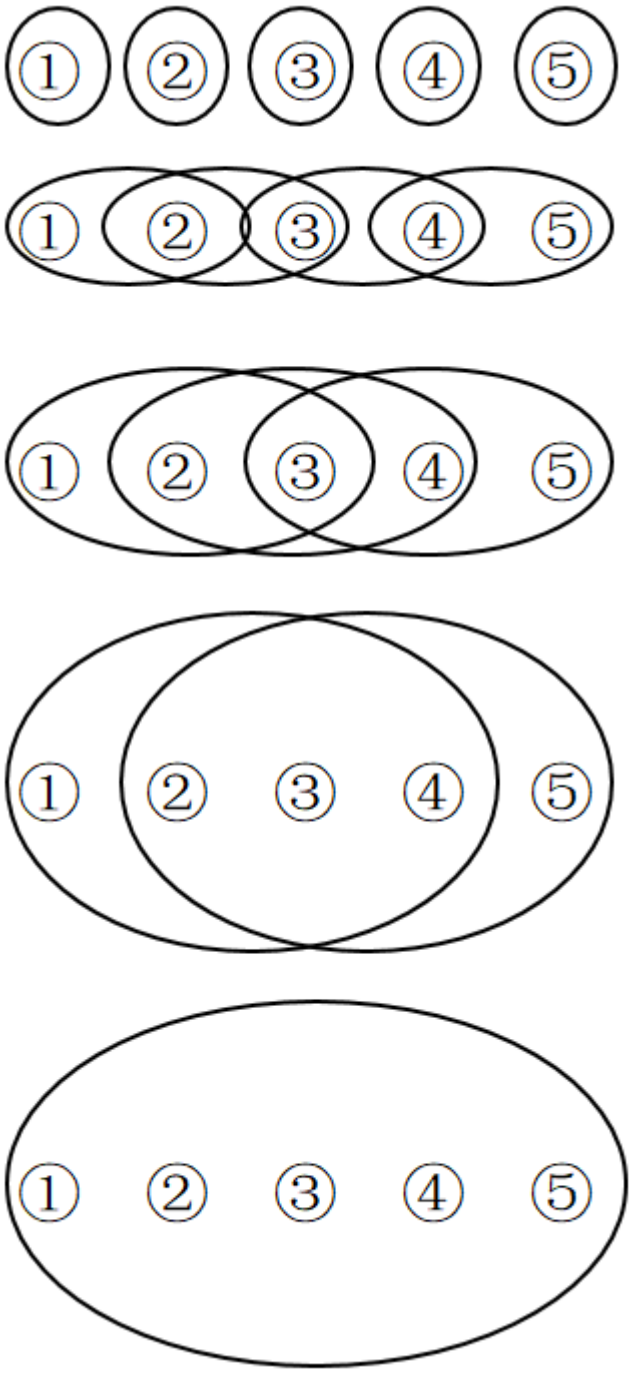


图18-2

```
#include <cstdio>
#include <algorithm>
using namespace std;
const int max_=1000000000;
int n,k,a[1500],dp[1500][1500][2];
int main(){
    scanf("%d%d",&n,&k);
    for(int i=1;i<=n;i++){
        scanf("%d",&a[i]);
    }
    a[++n]=k;
    sort(a+1,a+1+n);
    for(int i=1;i<=n;i++)
        if(a[i]==k) dp[i][i][0]=dp[i][i][1]=0;
        else dp[i][i][0]=dp[i][i][1]=max_;
```

```
    for(int L=2;L<=n;L++)
        for(int i=1;i<=n-L+1;i++){
            int j=i+L-1;
            dp[i][j][0]=min(dp[i+1][j][0]+(n-j+i)*(a[i+1]-a[i]),dp[i+1][j][1]+
(n-j+i)*(a[j]-a[i]));
            dp[i][j][1]=min(dp[i][j-1][1]+(n-j+i)*(a[j]-a[j-1]),dp[i][j-1][0]+
(n-j+i)*(a[j]-a[i]));
        }
    printf("%d\n",min(dp[1][n][0],dp[1][n][1]));
}
```

另一种递推写法代码

如图所示，这种方式写法比较简单，但理解稍难，你只要注意到它区间求解的顺序还是满足区间依赖，保证大区间所依赖的小区间先于大区间求出来。

这种写法的区间求解顺序如图 $18-3$ 所示：

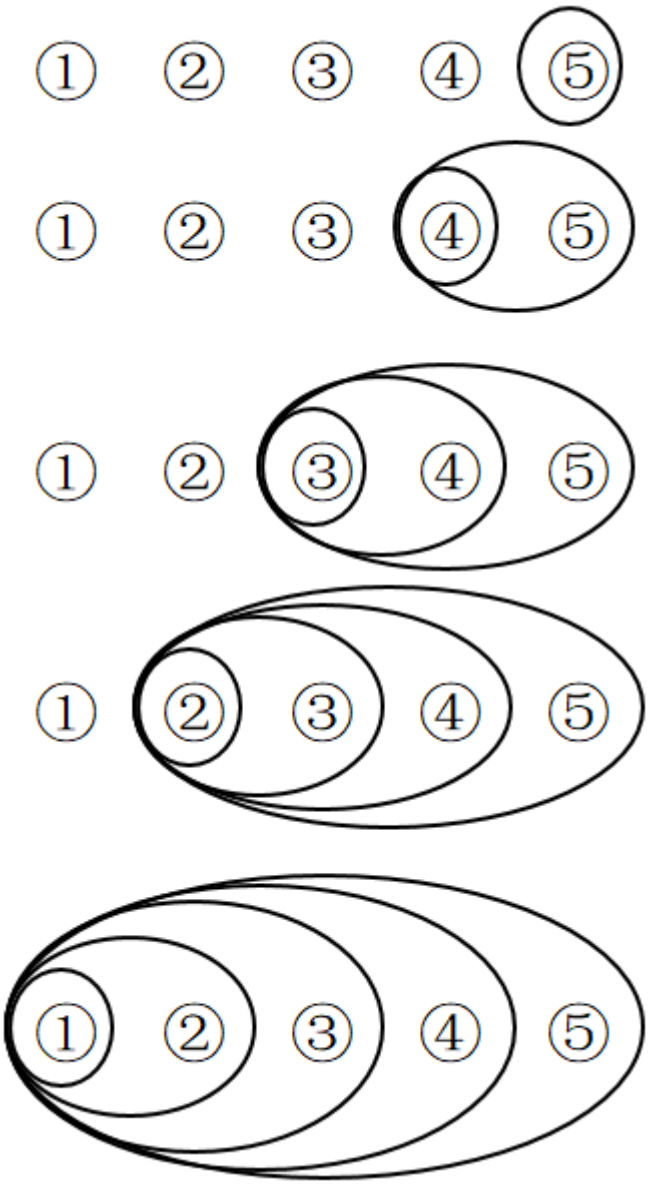


图18-3

```
#include<cstdio>
#include<algorithm>
using namespace std;
```

```
const int M=1e9;
int a[2100],dp[2100][2100][2];
int main(){
    int n,L;
    scanf("%d%d",&n,&L);
    for (int i=1;i<=n;i++)
        scanf("%d",&a[i]);
    a[++n]=L;
    sort(a+1,a+n+1);
    for (int i=1;i<=n;i++)
        if (a[i]==L) dp[i][i][0]=dp[i][i][1]=0;
        else dp[i][i][0]=dp[i][i][1]=M;
    for (int i=n;i>=1;i--)
        for (int j=i+1;j<=n;j++){
            dp[i][j][0]=min(dp[i+1][j][0]+(a[i+1]-a[i])*(n-j+i),dp[i+1][j][1]+
(a[j]-a[i])*(n-j+i));
            dp[i][j][1]=min(dp[i][j-1][1]+(a[j]-a[j-1])*(n-j+i),dp[i][j-1][0]+
(a[j]-a[i])*(n-j+i));
        }
    printf("%d\n",min(dp[1][n][0],dp[1][n][1]));
}
```

例题18-30：抽纸牌（POJ1651）

【题目描述】

给你n张纸牌，每张纸牌有个整数值，每次可以抽出除了第一张或最后一张的任意一张，产生的值是该张纸牌值乘以相邻两张的值，最终要求产生值尽可能小。

例如，5张纸牌值分别是10 1 50 20 5，如果按照这种顺序选1、20和50

产生的值如下

10 * 1 * 50 + 50 * 20 * 5 + 10 * 50 * 5 = 500 + 5000 + 2500 = 8000

如果以相反的顺序拿牌，即50、20和1，则值为
1 * 50 * 20 + 1 * 20 * 5 + 10 * 1 * 5 = 1000 + 100 + 50 = 1150。

【样例输入】

```
6
10 1 50 50 20 5
```

【样例输出】

```
3650
```

【分析】

和之前例题的区别一：某一段区间原有的元素最终都使用完了，本题则不然，样例中 1 号元素 10 和 5 号元素 5 最终还在，这就需要在状态转移时做些调整，大区间所依赖的两个子区间不能完全错开的，要有公共点。

和之前例题的区别二：这道题转移部分显得更加复杂，用 $dp[1][5]$ 描述 10 1 50 20 5，有如下三种方式

- $dp[1][2]$+$dp[2][5]$ 最后一次合并是 10 1 和 1 ~~50 20~~ 5（50 20在之前的合并中已经删除）
- $dp[1][3]$+$dp[3][5]$ 最后一次合并是 10 ~~1~~ 50 和 50 ~~20~~ 5（1 20在之前的合并中已经删除）
- $dp[1][4]$+$dp[4][5]$ 最后一次合并是10 ~~1 50~~ 20 和 20 5（1 50 在之前的合并中已经删除）

可以迁移到求单个状态 $dp[i][j]$ 的方法：最后一次合并的位置可以是 $i+1$ 到 $j-1$，需要 $O(n)$ 的时间

因此这道题目的**时间复杂度**为：**状态数量** $\times$ **单个状态转移次数** = $O(n^3)$

其他操作和之前都类似，只需要考虑状态、状态转移、初值、终值。

【解题思路】

状态：$dp[i][j]$

状态转移：$dp[i][j]$=$dp[i][k]$+$dp[k][j]$+$a[i]*a[k]*a[j](i+1 \le k \le j-1)$

初值：$dp[i][j]=MAX; dp[i][i]=0$; $dp[i][i+1]=0$

终值：$dp[1][n]$

时间复杂度：$O(n^2)$

【代码】

```
#include<algorithm>
#include<cstdio>
using namespace std;
const int max_=0x3fffffff;
int n,a[105],dp[105][105];
int main(){
    scanf("%d",&n);
    for(int i=1;i<=n;i++){
        scanf("%d",&a[i]);
    }
    for(int L=3;L<=n;L++)
        for(int i=1;i<=n-L+1;i++){
            int j=i+L-1;
            dp[i][j]=max_;
            for(int k=i+1;k<=j-1;k++)
            dp[i][j]=min(dp[i][j],dp[i][k]+dp[k][j]+a[i]*a[k]*a[j]);
        }
    printf("%d",dp[1][n]);
}
```

例题18-31：括号序列（POJ1141）

【题目描述】

求最少增加量使括号序列变规则，输出规则后的括号序列

规则的几种类型：

规则一：空是规则的

规则二：如果S是规则的，那么(S)[S]也是规则的

规则三：如果A和B是规则的，那么AB也是规则的

【样例输入】

```
([(]
```

【样例输出】

```
()[()]
```

【 分析】

和之前的例题相比，区别一：本题的转移方式没有那么明显，但仔细看合法的几种规则，状态转移方法不就在其中吗：规则一可视为初值；规则二可以得出最优值由左右两半区间得到；规则三括号两边括号脱离得到最优值。

和之前的例题相比，区别二：本题不但要求最优值，而且要输出最优值对应的括号序列，如何解决这一新问题呢？我们可以新增一个 p 数组，在状态转移时，记录添加括号的方案，输出的时候，利用 p 数组记录的值递归输出添加括号的方案。

根据上述分析，我们可以设计状态 $dp[i][j]$ ，它表示把从第 i 到第 j 的括号变为合法需要的最少括号添加量，再引入数组 $p[i][j]$ 表示第 i 到第 j 的区间得到最小添加量时的添加方案，解题时我们可以通过具体的样例来分析一些细节，进一步验证思路的正确性，比如这个样例“ ()[[]”，它的最优值用 $dp[1][5]$ 来表示，$dp[1][5]$ 可以由 $dp[1][2]+dp[3][5]$ 得来，与此同时也容易想到 $p[i][j]$ 可以记录他们的最优值来源的断开点 2，这是解决了左右半区间划分的方案记录问题，再来看到 $dp[3][5]$ 的求取，它可以由 $dp[4][4]$ 得来，括号就是剥离的一对括号，可以直接输出边界所在的括号，因此 p 数组不需要再记录具体的值，只要有剥离的信号即可，给它来个除了正常编号外的值就行，这里我们使用了 -1

总结：以样例"()[[]"为例，计算 dp 数组的同时，这样记录p数组： 执行 $dp[3][5]=dp[4][4]$ 的同时，执行 $p[3][5]=-1$ ；执行 $dp[1][5]=dp[1][2]+dp[3][5]$ 的同时，执行 $p[3][5]=-1$。有了这些分析，就可以设计出完整的解题思路了。

【解题思路】

状态

$dp[i][j]$表示最小添加量$p[i][j]$表示i到j的断开位置，分开的两部分都是规则的，如果是-1表示i和j括号匹配

状态转移

```
if (a[i]==a[j]) dp[i][j]=dp[i+1][j-1],p[i][j]=-1;
if (dp[i][j]>dp[i][mid]+dp[mid+1][j])
    dp[i][j]=dp[i][mid]+dp[mid+1][j],p[i][j]=mid;
```

初值

$dp[i][i]=1$

因为输入中只有四种括号字符，所以增加另外半边括号就可以了

终值

$dp[1][n]$就是最少插入数量，但在本题输出环节已经没什么用了，它的作用是在状态转移时用来比较最优值

$p[1][n]$利用p数组递归输出最少插入量形成的括号序列，讨论几种情况：

- $p[i][j]==-1$ 表示去两端括号，继续递归 $dfs(i+1,j-1)$

- $p[i][j] == x$ 表示在 x 位置插入括号，继续递归 $dfs(i,x)$，$dfs(x+1,j)$
- 遇到递归终点 $i == j$ 和 $i > j$

时间复杂度

$O(n^3)$

【代码】

```
#include <cstdio>
#include <algorithm>
#include <cstring>
using namespace std;
const int max_=1000000000;
char a[120],b[120];
int dp[120][120],p[120][120];
bool match(int i,int j){
    if (a[i]=='(' && a[j]==')') return 1;
    if (a[i]=='['&& a[j]==']') return 1;
    return 0;
}
void print_ans(int i,int j){
    if (i>j) return;
    if (i==j){
            if (a[i]=='('|| a[i]==')')
            printf("()");
        else printf("[]");
    }
    else if (p[i][j]==-1){
        printf("%c",a[i]);
        print_ans(i+1,j-1);
        printf("%c",a[j]);
    }
    else{
        print_ans(i,p[i][j]);
        print_ans(p[i][j]+1,j);
    }
}
int main(){
    gets(b);
    int n=strlen(b);
    for (int i=1;i<=n;i++)
        a[i]=b[i-1];
    for (int i=1;i<=n;i++)
         dp[i][i]=1;

    for(int L=2;L<=n;L++)
        for(int i=1;i<=n-L+1;i++){
            int j=i+L-1;
            dp[i][j]=max_;
            if (match(i,j)) {
                if (dp[i][j]>dp[i+1][j-1]){
                    dp[i][j]=dp[i+1][j-1];
                    p[i][j]=-1;
                }
            }
            for (int k=i;k<=j-1;k++){
                if (dp[i][j]>dp[i][k]+dp[k+1][j]) {
```

```
                    dp[i][j]=dp[i][k]+dp[k+1][j];
                    p[i][j]=k;
                }
            }
        }
    print_ans(1,n);
    printf("\n");
}
```

例题18-32：一的合并问题（zhima1112）

【题目描述】

在黑板上写了 n（$n \le 50$）个 1，进行如下操作：每次擦去其中两个数设为a、b，并写上数 $a \times b + 1$，重复这样的步骤，直至剩下最后一个数 A。请你编程序求出 A 的最大值，因为值过大，结果对 10007 取模。

【样例输入】

4

【样例输出】

5

【分析】

用解决之前题目的思路，很自然的进行如下设计：

状态 $dp[i][j]$ 表示从 i 到 j 最终合并成一个值后能获得的最大值，状态转移就是 $dp[i][j] = dp[i][k] * dp[k+1][j] + 1(i <= k <= j-1)$，于是时间复杂度就是 $O(n^3)$ 。

【代码】

```
#include<bits/stdc++.h>
using namespace std;
int f[1000][1000];
int main(){
    int n;
    scanf("%d",&n);
    for (int i=1;i<=n;i++) f[i][i]=1;
    for (int L=2;L<=n;L++)
        for (int i=1;i<=n-L+1;i++){
            int j=i+L-1;
            for (int k=i;k<=j-1;k++)
                f[i][j]=max(f[i][j],f[i][k]*f[k+1][j]+1);
        }
    printf("%d\n",f[1][n]);
}
```

仔细分析问题，对于任意一段同等长度的区间，例如 1 到 i 和 2 到 $i+1$ 这两段都是 $i-1$ 个 1 ，它们合并的结果自然也相同，所以状态只需要描述长度而不需要描述头尾位置，因此构造**状态** $f[i]$ 表示 i 个 1 经过合并能够产生的最大值，这样遍历区间就省下一个维度，时间复杂度降为 $O(n^2)$。

【解题思路】

状态转移： $f[i] = max(f[j] * f[i-j] + 1)(1 <= j <= i-1)$

初值： $f[1] = 1$

终值： $f[n]$

时间复杂度： $O(n^2)$

【代码】

```
#include<bits/stdc++.h>
using namespace std;
int f[20000];
int main(){
    int n;
    scanf("%d",&n);
    f[1]=1;
    for (int i=1;i<=n;i++)
        for (int j=1;j<=i-1;j++){
            f[i]=max(f[i]f[j]*f[i-j]+1);
        }
    printf("%d\n",f[n]);
}
```

前五道例题，带大家熟悉了区间DP的一般解题思路，现在给大家一道练手题巩固下。

例题18-33：合并 (luogu 3146 USACO16OPEN)

【题目描述】

N 正整数（$2 \le N \le 248$），每个数的取值范围是 $1 \ldots 40$，值相等的相邻数字可以合并成值大1的数，例如，两个

相邻的 7 可以变成一个 8 。求 N 个数最终能合并成一个最大的数的值。

【样例输入】

```
4
1
1
1
2
```

【样例输出】

```
3
```

【样例解释】

样例中，Bessie 首先将第二、三个 1 合并得到序列 $1\ 2\ 2$，然后将后两个 2 合并为 3 。

【解题思路】

状态描述$dp[i][j]$表示从 i 到 j 能合并成的最大数

转移：$dp[i][j] = dp[i][k] + dp[k+1][j] (i \le k \le j)$

初值：$dp[i][i] = a[i]$

终值：$dp[i][j]$ 中的最大值

时间复杂度：$O(n^3)$

例题18-34：零食出售（luogu 2858）

1S/128M

【题目描述】

约翰经常给产奶量高的奶牛发特殊津贴，于是很快奶牛们拥有了大笔不知该怎么花的钱．为此，约翰购置了N(1≤N≤2000)份美味的零食来卖给奶牛们．每天约翰售出一份零食．当然约翰希望这些零食全部售出后能得到最大的收益．这些零食有以下这些有趣的特性：

•零食按照1．．N编号，它们被排成一列放在一个很长的盒子里．盒子的两端都有开口，约翰每

天可以从盒子的任一端取出最外面的一个．

•与美酒与好吃的奶酪相似，这些零食储存得越久就越好吃．当然，这样约翰就可以把它们卖出更高的价钱．

•每份零食的初始价值不一定相同．约翰进货时，第i份零食的初始价值为$v_i(1 \le v_i \le 1000)$．

•第i份零食如果在被买进后的第a天出售，则它的售价是$v_i \times a$．

v_i的是从盒子顶端往下的第i份零食的初始价值．约翰告诉了你所有零食的初始价值，并希望你能帮他计算一下，在这些零食全被卖出后，他最多能得到多少钱．

【输入】

N

v_i

【输出】

能获得最多钱数

【输入#1】

```
5
1
3
1
5
2
```

【输出#1】

```
43
```

【样例解释】

可以按照编号 $1,5,2,3,4$ 的顺序卖掉零食就能获得 $1\times1+2\times2+3\times3+4\times1+5\times5=43$。

【分析】

用之前的例题思路就可以顺利解决此题了，不再赘述，细节可以参见代码。

【代码】

```
int n,sum,dp[2100][2100],a[2100];
int main(){
    scanf("%d",&n);
    for (int i=1;i<=n;i++){
```

```
        scanf("%d",&a[i]);
        sum+=a[i];
        dp[i][i]=a[i]*n;
    }
    for (int L=2;L<=n;L++)
        for (int i=1;i<=n-L+1;i++)
            {
                int j=i+L-1;
                dp[i][j]=max(dp[i][j],dp[i+1][j]+a[i]*(n-j+i));
                dp[i][j]=max(dp[i][j],dp[i][j-1]+a[j]*(n-j+i));
            }
    printf("%d\n",dp[1][n]);
}
```

但很多动态规划的题目，状态的设计并不像前五套例题那么容易，需要一番思考，我们来挑战下：

例题18-35：舞台（HDU4283）

【题目描述】

有个电视节目，你是总导演，节目内容是 n 个男生站成一排，一个个上台。每个男孩都希望尽早上台亮相，每个男孩有个不快乐指数D，不快乐值计算方式是（$k-1$）$*D$，k 表示第几个上场，因为他必须等待 $(k-1)$ 人。幸运的是，在小礼堂中有一间暗室，因此你可以将男孩暂时放到暗室中，并让身后的男孩在他之前登上舞台。由于暗室非常狭窄，因此首先进入暗室的男孩必须最后离开。你可以通过暗室改变男孩上台顺序，使不快乐值的总和最少。

【输入】

多组数据

$n(0<n<=100)$

$(0<=Di<=100)$

【输出】

一个整数，表示题目要求的最小值

【样例输入】

```
2
5
1
2
3
4
5
5
5
4
3
2
2
```

【样例输出】

```
Case #1: 20
Case #2: 24
```

【分析】

暗室的先进后出特点让你想到什么，利用它能改变出场顺序，否则这道题就是简单模拟题了，尝试按照区间DP的思路来做，先尝试**状态**描述：$dp[i][j]$表示从第i个到第j个男孩的最小不快乐值，但是该状态如何求取没有那么明显了，我们可以在纸上打个草稿，暗室可以把男孩按照进场先后次序进行划分：

$1 \ldots i \ldots n$

$1 \ldots i$ 进入暗室并比 $i+1$ 到 n 的男孩先登场，则 1 是 1 到 i 中最晚登台的男孩，所以区间划分如下：

第一波：2 到 i

第二波：1

第三波：$i+1$ 到 n

于是 $dp[1][n] = dp[2][i] + dp[i+1][n] + \ldots$

$2..i$ 的男孩们和 $i+1..n$ 的男孩们的最小不快乐值就是小区间子问题，子问题内部成员登场先后次序对大问题没有任何影响了，这就符合之前提到过的**无后效性**原则，那省略号部分应该填什么？

考虑各区域间作用产生的不快乐值即可

- 1 号男孩相对于 $2..i$ 产生的不快乐值 $a[i] \times (i-1)$
- $i+1$ 到 n 相对于 $1..i$ 产生的不快乐值 $(sum[n] - sum[i]) \times i$

形成一般性的**状态转移**：

$dp[i][j] = dp[i+1][k] + dp[k+1][j] + cost[i][i] \times (k-i) + cost[k+1][j] \times (k-i+1)$

【初值】

$dp[i][j] = max$ //初值的设定必须要避开所有可能的解

【终值】

$dp[1][n]$

【时间复杂度】

$O(n^3)$

【代码】

```
#include<bits/stdc++.h>
using namespace std;
const int max_=1000000000;
int n,T,a[110],dp[110][110],sum[110];
int main(){
    scanf("%d",&T);
    for(int t=1;t<=T;t++){
        scanf("%d",&n);
        for(int i=1;i<=n;i++){
            scanf("%d",&a[i]);
            sum[i]=sum[i-1]+a[i];
        }
        for(int L=2;L<=n;L++)
            for(int i=1;i<=n-L+1;i++){
                int j=i+L-1;
                dp[i][j]=max_;
                for(int k=i;k<=j;k++){
```

```
                    int tmp=dp[i+1][k]+dp[k+1][j]+a[i]*(k-i)+(sum[j]-sum[k])*(k-
i+1);
                    if (dp[i][j]>tmp) dp[i][j]=tmp;
                }
            }
        printf("Case #%d: %d\n",t,dp[1][n]);
    }
}
```

例题18-36：括号上色（CF149D）

【题目描述】

求依据一定的规则给一个合法的括号序列上色的方案总数，对10^9+7取模。

规则如下：

- 每个括号有三种情况：不上色、上红色、上蓝色
- 匹配的括号有且仅有一个上色
- 相邻的括号不能是相同颜色，两个都不上色可以

【输入】

s ($2 \le |s| \le 700$)

【输出】

一个整数，表示方案总数

【样例输入#1】

```
(())
```

【样例输出#1】

```
12
```

【样例输入#2】

```
(()())
```

【样例输出#2】

```
40
```

【样例输入#3】

```
()
```

【样例输出#3】

```
4
```

【样例解释】

样例1中可行的方案

样例1中不可行的方案

【分析】

继例题 $18-4$ 之后，又一道关于括号的题目，这道题目的状态表示很明显，但是括号多了颜色这一属性，这就导致转移时要思考的问题变多，另外左右划分区间的操作在这里没必要枚举所有断开点，其实只要找到左边界括号就近的右括号即可，想想这是为什么？

状态：$dp[i][j]$ 表示给从 i 到 j 的括号涂色的方案总数

转移：

分别考虑匹配括号、相邻括号的颜色要求

对于状态 $dp[i][j]$

- 如果 $a[i]$ 和 $a[j]$ 正好匹配的，则累加 $dp[i+1][j-1]$
- 如果 $a[i]$ 和 $a[j]$ 不匹配，则累加两端区间dp值的乘积

注意变量类型

如何快速找到某个左括号匹配的右括号，换言之就是右侧就近的右括号，你能想到什么？是不是很自然地想到栈这个基础数据结构，于是我们可以如下进行操作：

```
void matching(int n){
    stack<int>z;
    int tmp;
    for(int i=1;i<=n;i++){
        if (a[i]=='(') z.push(i);
        else {
            tmp=z.top();
            match[tmp]=i;
            z.pop();
        }
    }
}
```

利用栈操作我们成功地把括号配对情况记录在了match数组中，于是状态转移也容易设计了：

```
if (match[i]==j)dp[i][j]=(dp[i][j]+dp[i+1][j-1])% MOD;
else dp[i][j]=(dp[i][j]+dp[i][k]*dp[k+1][j])%MOD;
```

但我们发现 $dp[i][j]$ 根本无法描述颜色的情况，不符合无后效性的原则，i 和 j 位置的括号颜色对后续状态显然会产生影响，因此我们想到要改变状态描述，增加两个参数，如下：

```
dp[i][j][p][q]//后两个表示区间左右端点括号的颜色
初值就是dp[i][i+1][0][1]=dp[i][i+1][1][0]=dp[i][i+1][0][2]=dp[i][i+1][2][0]=1;
```

在此基础上再思考如何进行状态转移，两边去括号和分段分两半小区间分别怎么来写？

```
//两边去括号
if(match[i]==j) {
    for(int p=0;p<3;p++)
        for(int q=0;q<3;q++){
            if (q!=1) dp[i][j][0][1]=(dp[i][j][0][1]+dp[i+1][j-1][p][q])%MOD;
            if (p!=1) dp[i][j][1][0]=(dp[i][j][1][0]+dp[i+1][j-1][p][q])%MOD;
            if (q!=2) dp[i][j][0][2]=(dp[i][j][0][2]+dp[i+1][j-1][p][q])%MOD;
            if (p!=2) dp[i][j][2][0]=(dp[i][j][2][0]+dp[i+1][j-1][p][q])%MOD;
        }
}
else{
//分半，分成左右两半小区间
int k=match[i];
for(int x=0;x<3;x++)
   for(int y=0;y<3;y++)
        for(int p=0;p<3;p++)
            for(int q=0;q<3;q++)
                if (!(p==1&&q==1||p==2&&q==2))
                    dp[i][j][x][y]=(dp[i][j][x][y]+dp[i][k][x][p]*dp[k+1][j][q]
[y])%MOD;
}
```

【思考】

先来回答本题开篇提出的问题，为什么只要例举一种断开区间的方式，因为其他断开法都能被这种方法替代，以“(())(())(())”为例，断开位置有两种选择，在4、5之间或在8、9之间，第一次断开在8、9间，第二次再断开在4、5之间和反正顺序断效果完全等价，因此如果是左右半划分子区间就只要例举第一处断开点即可。

另外这道题目更适合用记忆化搜索来写，自底向上递推写法用了300ms，记忆化搜索只用了60ms，想想为什么出现这种差想想为什么出现这种差异?

####例题18-37：矩阵取数游戏 (luogu1005 NOIP 2007 提高组)

【题目描述】

帅帅经常跟同学玩一个矩阵取数游戏：对于一个给定的 $n \times m$ 的矩阵，矩阵中的每个元素 $a_{i,j}$ 均为非负整数。游戏规则如下：

1. 每次取数时须从每行各取走一个元素，共 n 个。经过 m 次后取完矩阵内所有元素；
2. 每次取走的各个元素只能是该元素所在行的行首或行尾；
3. 每次取数都有一个得分值，为每行取数的得分之和，每行取数的得分 = 被取走的元素值 $\times 2^i$，其中 i 表示第 i 次取数（从 1 开始编号）；
4. 游戏结束总得分为 m 次取数得分之和。

帅帅想请你帮忙写一个程序，对于任意矩阵，可以求出取数后的最大得分。

【输入】

第一行为两个用空格隔开的整数 n 和 m。

第 $2 \sim n+1$ 行为 $n \times m$ 矩阵，其中每行有 m 个用单个空格隔开的非负整数。

【输出】

输出文件仅包含 1 行，为一个整数，即输入矩阵取数后的最大得分。

【输入样例】

```
2 3
1 2 3
3 4 2
```

【输出样例】

```
82
```

【数据范围】

60%的数据满足：$1 \le n, m \le 30$ $1 \le n, m \le 30$，答案不超过10^{16}。

100%的数据满足：$1 \le n, m \le 80$ $1 \le n, m \le 80$，$0 \le a_{i,j} \le 1000$

【分析】

在正规考试中往往会在区间DP中夹杂着一些细节操作，比如超出 $longlong$ 类型的大整型数，我们就可以用高精度算法辅助。

这道题的动态规划思路和之前例题类似，虽然看似复杂了，其实就是由单行问题变成了多行问题，并且多行在求解时互不影响，每行单独求解，最后加起来即可。

每行的 dp 思路

【状态】

```
dp[i][j]表示还剩下从i到j这些数时能取得的最大值
```

【状态转移】(每行m列)

```
dp[i][j]=max(dp[i][j],dp[i+1][j]+a[i]*2^(m-j+i));
dp[i][j]=max(dp[i][j],dp[i][j-1]+a[j]*2^(m-j+i));
```

【状态遍历顺序】（有多种方式，这里提供一种）

```
for (int L=2;L<=m;L++)
    for (int i=1;i<=m-L+1;i++){
        int j=i+L-1;
}
```

【初值】

```
dp[i][i]=a[i]*(2^m)
```

【终值】

```
dp[1][m]
```

关注到数据范围，这样只能拿到60%的分数，需要高精度~~或者__int128~~，由于int128在某些比赛还不支持，并且高精度适用范围更广，通用性更强，对算法锻炼上更有价值。

【代码】

```
#include<bits/stdc++.h>
using namespace std;
const int q=10000;
const int maxn=100;
int n,m,a[maxn];
struct GJ{     //高精度结构体
    int w[50],len;
    GJ(){
        memset(w,0,sizeof(w));
        len=0;
    }  //赋初值
    void print(){ //结构体的输出过程，压位存储，可以提升速度
        printf("%d",w[len]);
        for (int i=len-1;i>=1;i--)
        {
            if (w[i]==0)
                printf("0000");
            else if (w[i]<10)
                printf("000");
            else if (w[i]<100)
                printf("00");
            else if (w[i]<1000)
                printf("0");
            printf("%d",w[i]);
        }
        printf("\n");
    }
}f[maxn][maxn],p2[maxn],ans;//f数组就是每行的最大值，p2是2的指数函数，ans最终的解

GJ max (const GJ &a,const GJ &b){//高精和高精比较最大值
```

```
    if (a.len>b.len)
        return a;
    else if (a.len < b.len)
        return b;
    for (int i=a.len;i>=1;i--)
        if (a.w[i]>b.w[i])
            return a;
        else if (a.w[i]<b.w[i])
            return b;
    return a;
}
GJ operator + (const GJ &a,const GJ &b) //高精+高精
{
    GJ c;
    c.len=max(a.len,b.len);
    int d=0;
    for (int i=1;i<=c.len;i++){
        c.w[i]=a.w[i]+b.w[i]+d;
        d=c.w[i] / q;
        c.w[i]=c.w[i] % q;
    }
    if (d) c.w[++c.len]=d;
    return c;
}
GJ operator * (const GJ &a,const int &b){  //高精*单精
    GJ c;
    c.len=a.len;
    int d=0;
    for (int i=1;i<=c.len;i++){
        c.w[i]=a.w[i]*b+d;
        d=c.w[i] / q;
        c.w[i]=c.w[i] % q;
    }
    while (d) {
        c.w[++c.len]=d % q;
        d = d / q;
    }
    return c;
}

void pow2(){   //2的指数函数预处理
    p2[0].w[1]=1;
    p2[0].len=1;
    for (int i=1;i<=m+2;i++){
        p2[i]=p2[i-1]*2;
    }
}
int main(){
     scanf("%d%d",&n,&m);
     pow2();
     for (int t=1;t<=n;t++){
        memset(f,0,sizeof(f));//多行共用f函数，重置
        for (int i=1;i<=m;i++){
            scanf("%d",&a[i]);
            f[i][i]=p2[m]*a[i];//重置
         }
        for (int L=2;L<=m;L++)
            for (int i=1;i<=m-L+1;i++){
```

```
                int j=i+L-1;
                f[i][j]=max(f[i][j],f[i+1][j]+p2[m-j+i]*a[i]);
                f[i][j]=max(f[i][j],f[i][j-1]+p2[m-j+i]*a[j]);
            }
        ans=ans+f[1][m];

    }
    ans.print();
}
```

例题18-38：取数（POJ2181）

1S/64M

【题目描述】

从一列数字中按照编号从小到大任意取数，第奇数个取到的数字就加，否则减，求能取到的最大总和。
【输入】

$n\ (1 \le n \le 150000)$ 表示一列数字的个数

$a_i(1 \le a_i \le 500)$

【输出】

最大值

【输入#1】

```
8
7
2
1
8
4
3
5
6
```

【输出#1】

```
17
```

【分析】

线性DP可以认为是区间DP的一种特殊形式，一个线段表示成一个点，比如这道题，我们可以把 $dp[i]$ 描述成最后一个数取第 i 个所能获得的最优值，但是仅仅这样描述不太够，因为不知道第 i 个数到底是第奇数个被选到的，还是第偶数个被选到的，所以我们可以在原数组中加一个参数 $dp[i][0..1]$，或者干脆用两个数组 $dp1[i]$ 、$dp2[i]$ 分别表示第奇数个被选到和第偶数个被选到的最优值。

接着设计状态转移， $dp1[i]$ 、$dp2[i]$ 的最优值显然是由 $dp1[1..i-1]$ 和 $dp2[1..i-1]$ 过来的，显然整个求解过程时间复杂度会到达 $O(n^2)$ ，但关注 n 的范围，这样会超时，有没有更快的办法? 看下图 $18-4$ ，我们可以再增加一个数组它把dp2[i]前缀的最大值保留下来，这样在状态转移时就省下了 $O(n)$ 的时间，时间复杂度降为 $O(n^2)$

dp1[i]	8	7	6	5	15	7	20
	8	7	5	2	9	1	6
dp2[i]	0	1	3	6	-1	14	11
max(dp2[1]..dp2[i])	0	1	3	6	6	14	14

如此优化后的状态转移如下：

$dp1[i] = max(dp1[i-1], dp2[i-1] + a[i])$

$dp2[i] = max(dp2[i-1], dp1[i-1] + a[i])$

【代码】

```
#define N 200000
int a[N];
int dp[N][2];
int n;
int main()
{
    scanf("%d", &n);
    for(int i = 1; i <= n; ++i)
        scanf("%d", &a[i]);
    for(int i = 1; i <= n; ++i)
    {
        dp[i][0] = max(dp[i-1][0], dp[i-1][1] + a[i]);//第奇数个选
        dp[i][1] = max(dp[i-1][1], dp[i-1][0] - a[i]);//第偶数个选
    }
    printf("%d\n", max(dp[n][0], dp[n][1]));
    return 0;
}
```

####例题18-39：合唱队（luogu 3205 HNOI 2010）

1S/128M

【题目描述】

$n\ (n \le 1000)$ 个人 $a_1, a_2, \ldots, a_n$ 拥有身高 $h_1, h_2, \ldots, h_n\ (1000 \le h_i \le 2000)$ 排队形，一个个进到新队形中，进队规则如下：

- 第一个人 a_1 直接插入空的当前队形中
- 对从第二个人开始的每个人 a_i ，如果比前面入队的 a_{i-1} 高，那么他直接站到当前队形的最右边，如果他比前面入队的 a_{i-1} 矮，那么将他直接站到当前队形的最左边。

例如，身高为 $1850, 1900, 1700, 1650, 1800, 1750$ 的六个人依次入队，最终的队形是 $1750, 1650, 1700, 1850, 1900, 1800$

【输入】

n

n个人的身高

【输出】

题意中的种数 mod 19650827

【数据范围】

对于 100% 的数据，$n \le 1000$，$1000 \le h_i \le 2000$。

【样例输入】

```
4
1701 1702 1703 1704
```

【样例输出】

```
8
```

【分析】

只要根据样例接合题目中的规则模拟下，发现其实就是牛吃草类似的问题，因为只能在两端操作，并且需要知道上一次到底在左边还是右边的痕迹，因此想到**状态**：

$dp[i][j][0]$ 表示第i个是最后一个加入的的方案数

$dp[i][j][1]$ 表示第j个是最后一个加入的的方案数

状态转移时，需要根据题意分类讨论边界元素的大小关系，其他细节大家可以自行思考，包括同余定理性质：(a+b)%c==(a%c+b%c)%c//加减乘同理，这里提供参考代码：

```
const int MOD=19650827;
int a[1100],dp[1100][1100][2],n;
int main(){
    scanf("%d",&n);
    for (int i=1;i<=n;i++)
        scanf("%d",&a[i]);
    for (int i=1;i<=n;i++)
        dp[i][i][0]=1;
    for (int L=2;L<=n;L++){
        for (int i=1;i<=n-L+1;i++){
            int j=i+L-1;
            if (a[i]<a[i+1])
                dp[i][j][0]=(dp[i][j][0]+dp[i+1][j][0])%MOD;
            if (a[i]<a[j])
                dp[i][j][0]=(dp[i][j][0]+dp[i+1][j][1])%MOD;
            if (a[j]>a[j-1])
                dp[i][j][1]=(dp[i][j][1]+dp[i][j-1][1])%MOD;
            if (a[j]>a[i])
                dp[i][j][1]=(dp[i][j][1]+dp[i][j-1][0])%MOD;
        }
    }
    printf("%d\n",(dp[1][n][0]+dp[1][n][1])%MOD);
}
```

例题18-40：取数游戏（zhima520）

1S/256MB

【题目描述】

小明刚刚学习了数据结构中“队列”的相关知识，现在有一个包含N个整数的队列，他和朋友小华决定玩一个游戏：小明和小华轮流在队首或队尾取一个数，当队列为空时，游戏结束，谁拿到的数的和最大，谁就赢了。

我们知道小明和小华都是很聪明的人，他们都会采取最优的策略来取数，现在是小明先手取数，你知道他最多能得到的和是多少？

【输入】

第一行1个整数N

第2..N行，每行一个整数$a_i(1 \le a_i \le 5000)$，表示队列的第i个整数

【输出】

一行一个整数，表示小明最多得到的和

【输入#1】

```
4
30
25
10
35
```

【输出#1】

```
60
```

【样例解释】
小明先取35，小华取30，然后小明再取25，小华最后取10

【数据范围】

对于100%的数据 N <= 5000

【分析】

因为两个人都存在决策，会互相影响，因此可以分开来设计状态，求解的时候可以依赖。

构造**状态**如下：

$dp1[i][j]$ i到j区间，小明先取，小明最多能取到的值

$dp2[i][j]$ i到j区间，小华先取，小明最少能取到的值

状态转移可以参加代码：

```
int n,dp1[5100][5100],dp2[5100][5100],a[5100];
int main(){

    scanf("%d",&n);
    memset(dp2,0x3F,sizeof(dp2));
    for (int i=1;i<=n;i++){
        scanf("%d",&a[i]);
        dp1[i][i]=a[i]; //小明取完
        dp2[i][i]=0;
```

```
    }
    for (int L=2;L<=n;L++)
        for (int i=1;i<=n-L+1;i++)
            {
                int j=i+L-1;
                dp1[i][j]=max(dp1[i][j],dp2[i][j-1]+a[j]);
                dp1[i][j]=max(dp1[i][j],dp2[i+1][j]+a[i]);
                dp2[i][j]=min(dp2[i][j],dp1[i][j-1]);
                dp2[i][j]=min(dp2[i][j],dp1[i+1][j]);
            }
    printf("%d\n",dp1[1][n]);
}
```

例题18-41：战狼 （hdu5115）

0.5S/512MB

【题目描述】

有一排狼，每只狼有两个值，A 表示自身的战力值，B 表示给相邻狼的战力加成值。人杀死一只狼就会受到这只狼的 A 和这只狼相邻狼的伤害 B 的和。如果某位置的狼被杀，那么他对相邻狼的加成也就消失，同时相邻位置也发生变化。求人杀掉一排狼的最小代价。

【输入】

第一行 T，表示 T 组数据

第二行 n $(2 \le n \le 200)$，表示狼的个数

$A(0 \le A \le 10^5)$

$B(0 \le B \le 5 \times 10^4)$

【输出】

最小代价，见样例

【输入#1】

```
2
3
3 5 7
8 2 0
10
1 3 5 7 9 2 4 6 8 10
9 4 1 2 1 2 1 4 5 1
```

【输出#1】

```
Case #1: 17
Case #2: 74
```

【样例解释】

第一个样例中，从左往右消灭狼， 会产生5 + 5 + 7 = 17的伤害。

【分析】

和之前的例题区别在于，区间中最后一个消失（消灭的狼）的不一定是在边界处，而是任意一个位置，我们可以
对于每个区间，如何划分成小区间，看最后一头被消灭的狼在哪

模拟（灭狼次序）

8 3 7 1 2 6 4 5

~~8~~ 3 7 1 2 6 4 5

~~8~~ 3 ~~7~~ 1 2 6 4 5

~~8~~ 3 ~~7~~ 1 2 ~~6~~ 4 5

~~8~~ 3 ~~7~~ 1 2 ~~6~~ 4 ~~5~~

~~8~~ 3 ~~7~~ 1 2 ~~6~~ ~~4~~ ~~5~~

~~8~~ ~~3~~ ~~7~~ 1 ~~2~~ ~~6~~ ~~4~~ ~~5~~

~~8~~ ~~3~~ ~~7~~ ~~1~~ ~~2~~ ~~6~~ ~~4~~ ~~5~~

通过如上模拟，可以进行如下设计

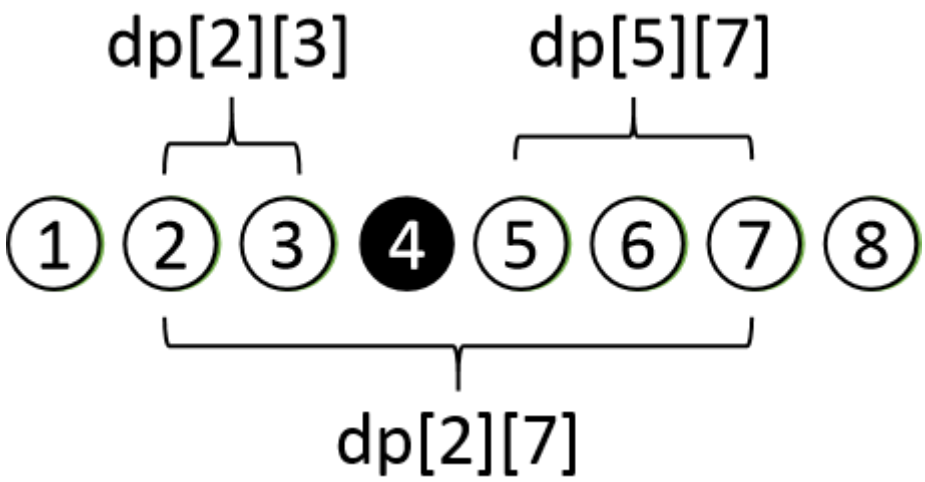
【状态】

$dp[i][j]$ 表示消灭从i到j的狼，需要承受的最小伤害

【状态转移】

$dp[i][j] = dp[i][k-1] + dp[k+1][j] + b[i-1] + a[k] + b[j+1]$

通过如下图进一步来解释这个状态转移，以某次划分为例，如下图




```
const int MAX=0x3FFFFFFF;
int T,n,dp[210][210],a[210],b[210];
int main(){
    scanf("%d",&T);
    for (int t=1;t<=T;t++)
    {
        scanf("%d",&n);
        for (int i=1;i<=n;i++)
            for (int j=i;j<=n;j++)
                dp[i][j]=MAX;
        for (int i=1;i<=n;i++)scanf("%d",&a[i]);
        for (int i=1;i<=n;i++)scanf("%d",&b[i]);
        for (int L=1;L<=n;L++)
```

```
            for (int i=1;i<=n-L+1;i++){
                int j=i+L-1;
                for (int k=i;k<=j;k++){
                    dp[i][j]=min(dp[i][j],dp[i][k-1]+dp[k+1][j]+b[i-
1]+a[k]+b[j+1]);
                }
            }
        printf("Case #%d: %d\n",t,dp[1][n]);
    }
```

例题18-42：三角形的最小划分(zhima961)

【题目描述】

N个顶点的凸多边形，各顶点权值已知，要求划分成N-2个三角形，使各三角形顶点权值乘积之和为最小
当n=4，各顶点的权值 分别为10,5,7,6时，所求最小
值为10*5*6+5*6*7=510。

【输入】

第一行：一个整数n(n<=200)
第二行：n个不超过1000的整数,依次表示各顶点的权值

【输出】

一个整数表示最小值

【样例输入】

```
4
10 5 7 6
```

【样例输出】

```
510
```

【分析】

本题看似是个几何图形，其实我们可以通过状态的制定把它看成是区间线段，利用i到j连续的顶点描述一个多边形，划分的时候需要在i到j中再寻找一个点k，利用i-j-k连线构成的三角形对大区间多边形进行划分。

顺着这个思路我们进行如下设计：

【状态】

$dp[i][j]$ 表示 i 到 j 连边组成的多边形通过合理划分能够产生的最小权值和

【状态转移】

$dp[i][j]$ = min($dp[i][k]$ + $dp[k][j]$ + $a[i]$ * $a[k]$ * $a[j]$) ($i+1 \le k \le j-1$)

【初值】

$dp[i][i+2]$ // 初值就是三角形

【终值】

$dp[1][n]$

【时间复杂度】

$O(n^3)$

【代码】

```
#include<bits/stdc++.h>
using namespace std;
int n,a[210];
long long dp[210][210];
const int MAX=0x3fffffff;
int main(){
    scanf("%d",&n);
    for (int i=1;i<=n;i++)
        scanf("%d",&a[i]);
    for (int i=1;i<=n-2;i++)
        dp[i][i+2]=a[i]*a[i+1]*a[i+2];
    for (int L=4;L<=n;L++)
        for (int i=1;i<=n-L+1;i++)
        {
            int j=i+L-1;
            dp[i][j]=MAX;
            for (int k=i+1;k<=j-1;k++){
                dp[i][j]=min(dp[i][j],dp[i][k]+dp[k][j]+a[i]*a[k]*a[j]);
            }
        }
    printf("%d",dp[1][n]);
}
```

例题18-43：涂色（洛谷4170）

假设你有一条长度为 5 的木版，初始时没有涂过任何颜色。你希望把它的 5 个单位长度分别涂上红、绿、蓝、绿、红色，用一个长度为 5 的字符串表示这个目标：`RGBGR`。

每次你可以把一段连续的木版涂成一个给定的颜色，后涂的颜色覆盖先涂的颜色。例如第一次把木版涂成 `RRRRR`，第二次涂成 `RGGGR`，第三次涂成 `RGBGR`，达到目标。

用尽量少的涂色次数达到目标。

【输入#1】

```
AAAAA
```

【输出#1】

```
1
```

【输入#2】

```
RGBGR
```

【输出#2】

```
3
```

【分析】

用区间DP设计思路，不难得出如下设计

【状态】

$dp[i][j]$ 表示从 i 到 j 涂色的最少次数

- 剥离//RGBGR
- 左右划分//RARCC

这两个配合使用，就是**【状态转移】**

```
剥离：if (s[i]==s[j]) dp[i][j]=min(dp[i+1][j],dp[i][j-1]);
左右划分：dp[i][j]=min(d[i][k],dp[k+1][j])(i<=k<=j-1)
```

【代码】

```
#include<bits/stdc++.h>
using namespace std;
const int MAX=1e9;
char a[55];
int dp[55][55];
int main(){
    scanf("%s",a+1);
    int len=strlen(a+1);
    for (int i=1;i<=len;i++) dp[i][i]=1;
    for (int L=2;L<=len;L++){
        for (int i=1;i<=len-L+1;i++){
            int j=i+L-1;
            dp[i][j]=MAX;
            if (a[i]==a[j])
                dp[i][j]=min(dp[i+1][j],dp[i][j-1]);
            for (int k=i;k<j;k++){
                dp[i][j]=min(dp[i][j],dp[i][k]+dp[k+1][j]);
            }
        }
    }
    printf("%d\n",dp[1][len]);
}
```

例题18-44：字符串改造（HDU2476）

【题目描述】

给出两个串s1和s2，一次只能将一个区间刷一次，问最少几次能让s1=s2

例如zzzzzfzzzzz，长度为11，我们就将下标看做0~10

先将0~10刷一次，变成aaaaaaaaaaa

1~9刷一次，abbbbbbbbba

2~8:abcccccccba

3~7:abcdddddcba

4~6:abcdeeedcab

5:abcdefedcab

这样就6次，变成了s2串了

第二个样例也一样

0

先将0~10刷一次，变成cccccccccccb

1~9刷一次，cddddddddcb

2~8:cdcccccccdcb

3~7:cdcdddddcdcb

4~6:cdcdcccdcdcb

5:cdcdcdcdcdcb

最后竟串尾未处理的刷一次

就变成了串2cdcdcdcdcdcd

所以一共7次

【样例输入】

```
zzzzzfzzzzz
abcdefedcba
abababababab
cdcdcdcdcdcd
```

【样例输出】

```
6
7
```

【分析】

这是例题 $18-14$ 的加强版本

回顾上一题，我们只要把一个未涂色的串涂成指定的颜色，而这一题是从已有颜色的串涂到指定颜色的串

和上一题区别：初始串 A，目标串 B，如果某个字符 $A[i]==B[i]$ ，则不一定要涂色

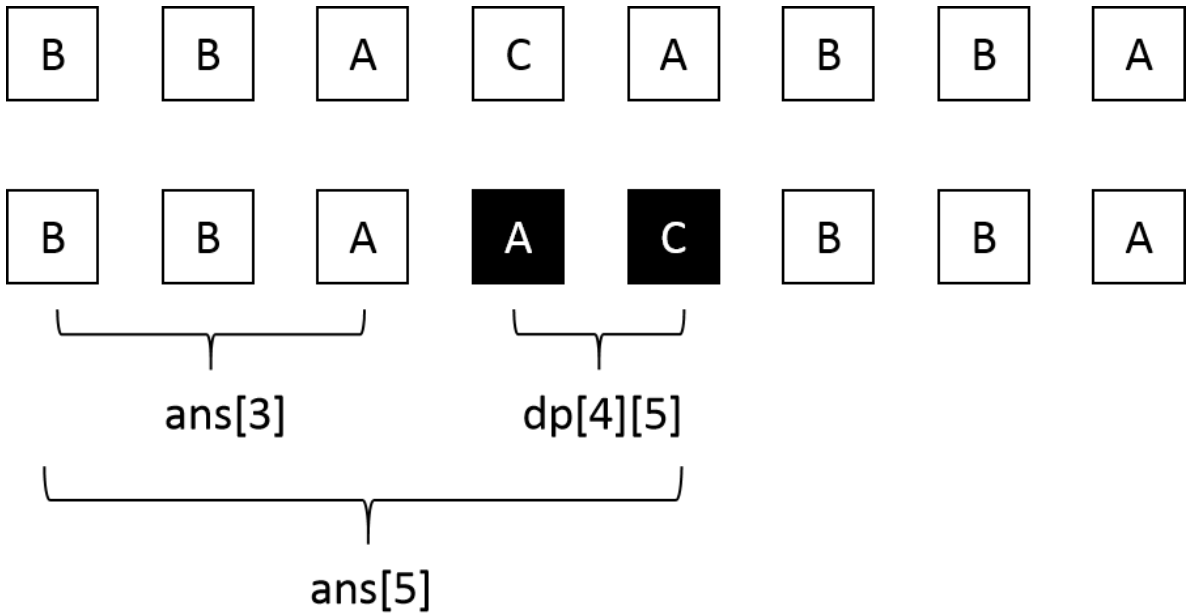
ZZZZZZZZZ
BBAACBBA

BBACABBA
BBAACBBA

看上述样例，对同样的目标串BBAACBBA，第一个初始串相当于空色串，因此是4，第二对串中，除了‘CA’和‘AC’其余都是相同的，我就只考虑如何从空色串变到'AC',这就调用上一题的dp思路即可

原有 $dp[i][j]$ 表示从空色串变到目标串需要的最少涂色次数

引入 $ans[i]$ 表示初始串和目标串前i个字符转换需要几次涂色



$if\ (A[i] == B[i])\ ans[i] = ans[i-1]$ // 第 i 个字符不用涂色，相当于前 $i-1$的问题了

$ans[i] = min(ans[k], dp[k+1][i])\ (1 \le k < i)$

【代码】

```
#include<bits/stdc++.h>
using namespace std;
char a[110],b[110];
int dp[110][110],ans[110];
const int MAX=1e9;
int main(){
    int  n;
    while (~scanf("%s%s",a+1,b+1)){
        n=strlen(a+1);
        for (int i=1;i<=n;i++) dp[i][i]=1;
        for (int L=2;L<=n;L++)
            for (int i=1;i<=n-L+1;i++){
                int j=i+L-1;
                dp[i][j]=MAX;
                if (b[i]==b[j])
                    dp[i][j]=min(dp[i+1][j],dp[i][j-1]);
                for (int k=i;k<j;k++)
                    dp[i][j]=min(dp[i][j],dp[i][k]+dp[k+1][j]);
            }
        for (int i=1;i<=n;i++){
            ans[i]=dp[1][i];
            if (a[i]==b[i]) ans[i]=min(ans[i],ans[i-1]);
            for (int j=1;j<=i-1;j++){
                ans[i]=min(ans[i],ans[j]+dp[j+1][i]);
            }
        }
        printf("%d\n",ans[n]);
    }
}
```

例题18-45：变装（LightOJ1422）

【题目描述】

小憨憨 参加圣诞节的一些派对，并且需要穿上对应派对的衣服，所以他需要多次换衣服，为了方便，他可以选择脱掉一些衣服或者穿上新衣服，比如说，他穿着 美队 的衣服，外面又穿着 钢铁侠 的衣服，当他要参加 美队 服装派对时，他可以选择脱掉 钢铁侠 的衣服（因为 钢铁侠 衣服的里面有 美队 的衣服），或者他可以在穿一件 美队 的衣服，小憨憨 是个爱干净的人，当他脱下 钢铁侠 的衣服后，如果需要再穿 钢铁侠 的衣服，他会选择再穿一件新的。（如果他先穿A衣服，又穿上B衣服，再穿一件C衣服，如果他想让最外面的衣服是A，他可以选择直接穿一件A，或者先把C脱掉，再把B脱掉）。

【输入】

整数 $T(1 \le T \le 200)$，表示数据组数

每个数据的第一行是整数 $N(1 \le N \le 100)$ 表示衣服数量，接下来一行表示 N 个派对（严格按照从左到右的顺序参加）需要的衣服的种类 $C_i(1 \le C_i \le 100)$

【输出】

按照样例格式输出最少需要的衣服数量

【输入#1】

```
2
4
1 2 1 2
7
1 2 1 1 3 2 1
```

【输出#1】

```
Case 1: 3
Case 2: 4
```

【分析】

和涂色其实一模一样，你发现了吗

1 ~~2 1 2~~

如果想通过脱衣服获得之前穿的那件，相当于一整段涂色

思路可以自行思考或者参见之前的例题

【代码】

```
#include<bits/stdc++.h>
using namespace std;
int T,n,dp[200][200],a[200];
int main(){
    scanf("%d",&T);
    for (int t=1;t<=T;t++){
        printf("Case %d: ",t);
        scanf("%d",&n);
        memset(dp,0x3F,sizeof(dp));
        for (int j=1;j<=n;j++) {
            scanf("%d",&a[j]);
            dp[j][j]=1;
        }
        for (int L=2;L<=n;L++)
```

```
            for (int i=1;i<=n-L+1;i++)
                {
                    int j=i+L-1;
                    if (a[i]==a[j])
                        dp[i][j]=min(dp[i][j-1],dp[i+1][j]);
                    for (int k=i;k<=j-1;k++)
                        dp[i][j]=min(dp[i][j],dp[i][k]+dp[k+1][j]);
                }
        printf("%d\n",dp[1][n]);
    }
}
```

练习题：括号（POJ2955）

题意

我们给括号序列下一个“合法”定义：

- 空序列是一个合法的
- 如果s是合法的，那么（s）和[s]也是合法的
- 如果a和b是合法的，那么ab也是合法的

例如以下都是合法括号序列

```
(), [], (()), ()[], ()[()]
```

而这些则不是

```
(, ], )(, ([)], ([(]
```

对于给定的括号序列，找一个最长的合法的括号子序列（子序列允许不连续）

样例输入

```
((()))
()()()
([]])
)[)(
([][][)
end
```

样例输出

```
6
6
4
0
6
```

UVA1625

【分析】
状态描述
把求解的东西打散
不同状态间的颜色增量

利用每种颜色在本串的起始点和终结点

```
#include<cstdio>
#include<cstring>
#include<algorithm>
#define INF 0x3f3f3f3f
#define LL long long
#define N 5005
using namespace std;
unsigned char c;
int dp[N][N], cnt[N][N];
int bf[100], bs[100], ef[100], es[100];
char f[N], s[N];

int main()
{
    int t;
    scanf("%d", &t);
    while (t--){
        scanf("%s %s", f+1, s+1);
        int lenf = strlen(f+1), lens = strlen(s+1);
        for (c = 'A'; c <= 'Z'; c++) bf[c] = bs[c] = INF, ef[c] = es[c] = 0;
        //bf[c]表示颜色c在第一个颜色序列的开始位置   first begin
        //bs[c]表示颜色c在第二个颜色序列的开始位置    second begin
        //ef[c]表示颜色c在第一个颜色序列的结束位置    first end
        //es[c]表示颜色c在第二个颜色序列的结束位置   second end
        for (int i = 1; i <= lenf; i++){
            c = f[i];
            bf[c] = min(i, bf[c]);
            ef[c] = i;
        }
        for (int i = 1; i <= lens; i++){
            c = s[i];
            bs[c] = min(i, bs[c]);
            es[c] = i;
        }
        for (int i = 0; i <= lenf; i++){
            for (int j = 0; j <= lens; j++){
                if (i){
                    cnt[i][j] = cnt[i-1][j];
                    c = f[i];
                    if (bf[c] == i && bs[c] > j) cnt[i][j]++;
                    if (ef[c] == i && es[c] <= j) cnt[i][j]--;
                }
                else if (j){
                    cnt[i][j] = cnt[i][j-1];
                    c = s[j];
                    if (bs[c] == j && bf[c] > i) cnt[i][j]++;
                    if (es[c] == j && ef[c] <= i) cnt[i][j]--;
                }
            }
```

```
        }
        for (int i = 0; i <= lenf; i++)
            for (int j = 0; j <= lens; j++){
                if (!i && !j) continue;
                dp[i][j] = INF;
                if (i) dp[i][j] = min(dp[i][j], dp[i-1][j]+cnt[i-1][j]);
                if (j) dp[i][j] = min(dp[i][j], dp[i][j-1]+cnt[i][j-1]);
            }
        printf("%d\n", dp[lenf][lens]);
    }
    return 0;
}
```

UVA12991

【问题描述】

一个 N 层的大楼，每层只有一个游戏室，可以设置一个乒乓球桌或游泳池。第 i 层有 $T**i$ 个人喜欢乒乓球和 $P**i$ 个人喜欢游泳。

现在要求使每个人到最近的喜欢的类型的活动室的距离的**和**最小，且这栋大楼要有至少一个乒乓球桌和至少一个游泳池。

这里的距离指楼层差的绝对值。例如，如果一个人到他喜欢的类型的游戏室位于同一楼层，则为 0；如果所需类型的最接近的游戏室正好在该员工的上方或下方一层，则为 1，以此类推。

【输入格式】

第一行给出数据组数 T（$1\le T\le 100$）。接下来对于每组数据，首先一行给出 N（$2\le N\le 4000$），代表该大楼一共有多少层；接下来 N 行第 i 行给出两个整数 $T**i,P**i$（$1\le T**i,P**i\le 109$），代表第 i 层的人数，意义同题意

【输出格式】

对于第 x 组（x 从 11 开始标号）数据的答案 y，在第 x 行输出 `Case #x: y`

【样例输入】

```
1
2
10 5
4 3
```

【样例输出】

```
Case #1: 9
```

[Game Rooms - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)]

四边形不等式优化DP

https://blog.csdn.net/AC__dream/article/details/123668489

完成这道题目：

[P1880 NOI1995] 石子合并 - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)

【参考代码】

```
#include<iostream>
#include<cstdio>
using namespace std;
int cnt[210],s[210][210],dp[210][210],n,temp,te,dp2[210][210],maxn,minn;
int main()
{
    scanf("%d",&n);
    for(int i=1;i<=n;i++)scanf("%d",&cnt[i]),cnt[i]+=cnt[i-1],s[i][i]=i,s[i+n]
[i+n]=i+n;
    for(int i=1;i<=n;i++)cnt[i+n]=cnt[i]+cnt[n];
    for(int i=n*2;i>=1;i--)
        for(int j=i+1;j<=n*2;j++)//参考第1点
        {
            temp=0x7fffffff;
            dp2[i][j]=max(dp2[i+1][j],dp2[i][j-1])+cnt[j]-cnt[i-1]; //参考第2点
            for(int k=s[i][j-1];k<=s[i+1][j];k++)
            {
                if(temp>dp[i][k]+dp[k+1][j]+cnt[j]-cnt[i-1])
                {
                    temp=dp[i][k]+dp[k+1][j]+cnt[j]-cnt[i-1];
                    te=k;
                }
            }
            dp[i][j]=temp;
            s[i][j]=te;
        }
    minn=0x7fffffff;
    for(int i=1;i<=n;i++)
    {
        minn=min(minn,dp[i][i+n-1]);
        maxn=max(maxn,dp2[i][i+n-1]);
    }
    printf("%d\n%d",minn,maxn);
    return 0;
}
```

Polygon

[P4342 IOI1998] Polygon - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)

Walk

【矩阵快速幂】

Walk - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)

矩阵乘法：

[矩阵乘法_百度百科 (baidu.com)](https://baidu.com)

斐波那契数列的矩阵优化：

https://blog.csdn.net/m0_53899788/article/details/129404634

18.3 小结

|区间DP的定义：以区间长度为阶段的DP问题

|时间复杂度：一般情况下可以看成状态数*状态转移时间

|特点：区间求取顺序可以有很多种，但只有一个原则，从小问题求到大问题，遵循依赖的关系

区间DP的实现有自底向上的递推写法和自顶向下的记忆化搜索写法，用哪种方法视具体题目而定